Lecture 2 revision sheet

Linear algebra 1: matrices, linear systems and when they can be solved

This sheet lists the definitions, formulas and results from Lecture 2, without derivations or discussion. These are in the full notes (`linear-algebra-1.ipynb` and the homework notebook); the grey pointers give the section. → L*n* marks material that later lectures build on. [*] marks material that is optional in the notes, [**] material that is advanced.

Key messages

1. Almost every method in the module needs the solution of a **linear system** $A\boldsymbol{x} = \boldsymbol{b}$ at some point, and it is often the most expensive step.
2. $A\boldsymbol{x}$ is a **weighted sum of the columns** of A, with weights x_j. Solving $A\boldsymbol{x} = \boldsymbol{b}$ means finding the weights that reach $\boldsymbol{b}$.
3. For a square matrix: $\det A \neq 0 \iff$ columns linearly independent $\iff$ A invertible $\iff$ a **unique** solution for every $\boldsymbol{b}$. Otherwise there is either *no* solution or *infinitely many*, depending on $\boldsymbol{b}$.
4. A matrix is a **linear transformation**: its columns are the images of the basis vectors, and $|\det A|$ is the factor by which it scales areas (volumes).
5. In code: use `@` (not `*`) for matrix products, compare floats with `np.allclose`, and solve systems with `sl.solve` rather than forming A^{-1}.

1 Notation and basic operations

Full notes: *Notation & terminology; Arithmetic and other simple matrix operations* → **used in L3, L4, L8**

Vectors	Bold, and *column* vectors by default: $\boldsymbol{x} = (x_1, \ldots, x_m)^T$ is $m \times 1$.
Matrices	$A = [a_{ij}]_{m\times n} \in \mathbb{R}^{m\times n}$: m rows, n columns (rows first). a_{ij} is in row i, column j; $\boldsymbol{a}_{:j}$ is the j-th column. Square if $m = n$. (Python indices start at 0.)
Equality, sum, scalar	$A = B \iff a_{ij} = b_{ij}$ for all i, j (same size); $A \pm B = [a_{ij} \pm b_{ij}]$; $\alpha A = [\alpha a_{ij}]$.
Transpose	$A^T = [a_{ji}]_{n\times m}$. Conjugate (Hermitian) transpose $A^H = \overline{A}^T$.
Special matrices	*Symmetric* $A = A^T$; *skew-symmetric* $A = -A^T$ (zero diagonal); *Hermitian* $A = A^H$. Identity I_n; diagonal; zero (null) matrix.

- **Transpose rules:** $(A + B)^T = A^T + B^T$, $(\alpha A)^T = \alpha A^T$, $(A^T)^T = A$, and $(AB)^T = B^T A^T$ (order reverses; so $(ABC)^T = C^T B^T A^T$).
- **Symmetric results** [*]: if A is symmetric then BAB^T is symmetric for any conformable B. If A, B are symmetric then $AB + BA$ is symmetric and $AB - BA$ is skew-symmetric.

`A @ B`, `A @ x`	matrix product (also `np.dot`, `np.matmul`); `A*B` is *element-wise*
`np.allclose(A, B)`	equality to a tolerance; `np.array_equal` is exact and fails under round-off
`B = A` vs `np.copy(A)`	`B = A` (and `A.T`) share data with `A`; changing one changes the other
`A.T`, `np.conjugate(A).T`	transpose, conjugate transpose
`np.shape`, `np.ndim`, `np.size`	shape (m, n); number of array axes (not the maths "dimension"); number of entries
`np.eye`, `np.diag`, `np.zeros`, `np.ones`	identity; extract or build diagonals (offset `k`); zeros; ones

2 Matrix–vector and matrix–matrix products

Full notes: *Matrix-vector multiplication; ... (again; code speed) [*]; Matrix-matrix product [*]* → **used in L3, L6, L7**

$$\boldsymbol{b} = A\boldsymbol{x}: \quad b_i = \sum_{j=1}^{n} a_{ij}x_j \quad (i = 1, \ldots, m), \qquad \text{equivalently} \qquad A\boldsymbol{x} = x_1\boldsymbol{a}_{:1} + x_2\boldsymbol{a}_{:2} + \cdots + x_n\boldsymbol{a}_{:n},$$

$$C = AB: \quad c_{ij} = \sum_{k=1}^{n} a_{ik}b_{kj}, \qquad (m \times n)(n \times p) = (m \times p) \quad \text{(inner sizes must match: "conformable").}$$

- **Linearity:** $A(\boldsymbol{x} + \boldsymbol{y}) = A\boldsymbol{x} + A\boldsymbol{y}$ and $A(\alpha\boldsymbol{x}) = \alpha A\boldsymbol{x}$.
- **Matrix products** are distributive, $A(B+C) = AB+AC$, and associative, $A(BC) = (AB)C$, and scalars factor out, $A(\alpha B) = \alpha AB$. They are **not commutative**: $AB \neq BA$ in general (matrices for which $AB = BA$ are said to *commute*).
- **Cost** [*]: from the loops, $A\boldsymbol{x}$ takes mn multiply–adds, i.e. $\mathcal{O}(n^2)$ for a square matrix, so doubling n quadruples the work; AB takes mnp, i.e. $\mathcal{O}(n^3)$ (Big-O, L1). Log–log timing plot: slope ≈ 2. NumPy's `@` does the same arithmetic in compiled code, so it is faster by a large constant factor. → **used in L7**

3 Linear systems

Full notes: *Motivation – linear (or matrix) systems; Row operations… [*]* → **used in L5, L7**

- m linear equations in n unknowns, $\sum_j a_{ij}x_j = b_i$, are the matrix system $A\boldsymbol{x} = \boldsymbol{b}$ with A $m \times n$, $\boldsymbol{x}$ $n \times 1$, $\boldsymbol{b}$ $m \times 1$. Often (not always) $m = n$.
- **Geometry:** each equation is a constraint, a line in 2D (plane in 3D). The solution is their intersection. In 2D the lines *cross* (unique solution), are *parallel* (no solution) or *coincide* (infinitely many solutions).
- **Example:** $2x + 3y = 7,\ x - 4y = 3$, i.e. $\begin{pmatrix} 2 & 3 \\ 1 & -4 \end{pmatrix}\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 7 \\ 3 \end{pmatrix}$, has the unique solution $(x, y) = (37/11,\ 1/11)$.
- **Row operations** (do not change the solution, applied to the augmented matrix $[A \,|\, \boldsymbol{b}]$): (1) multiply a row by a non-zero scalar; (2) add a multiple of one row to another; (3) swap rows. Reducing $[A \,|\, \boldsymbol{b} \,|\, I]$ to $[I \,|\, \boldsymbol{x} \,|\, A^{-1}]$ gives both the solution and the inverse. These ideas underlie Gaussian elimination and the LU decomposition.
- **Row echelon form (REF):** zero rows at the bottom, and each row's leading (first non-zero) entry strictly to the right of the one above. **Reduced** (RREF): in addition every leading entry is 1 and is the only non-zero in its column.

4 The matrix inverse

Full notes: *Matrix inverse; The inverse matrix; Singular matrices* → **used in L3, L4, L7**

- For square A, the inverse A^{-1} satisfies $AA^{-1} = A^{-1}A = I$. If it exists it is **unique**. (For non-square A there can be one-sided *left* or *right* inverses.)
- **Properties** (for invertible A, B and $\alpha \neq 0$): $(\alpha A)^{-1} = \alpha^{-1}A^{-1}$, $(AB)^{-1} = B^{-1}A^{-1}$, $(A^{-1})^{-1} = A$, $(A^T)^{-1} = (A^{-1})^T$.
- If A^{-1} exists, $A\boldsymbol{x} = \boldsymbol{b} \Rightarrow \boldsymbol{x} = A^{-1}\boldsymbol{b}$.
- A matrix with $\det A = 0$ is **singular** and has no inverse (`sl.inv` raises a `LinAlgError`).

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}: \quad A^{-1} = \frac{1}{ad - bc}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \ (ad - bc \neq 0); \qquad \text{in general} \quad A^{-1} = \frac{1}{\det A}\operatorname{adj}(A),$$

where the *adjugate* $\operatorname{adj}(A) = C^T$ and C is the cofactor matrix (§5). Example: $\begin{pmatrix} 2 & 3 \\ 1 & -4 \end{pmatrix}^{-1} = \frac{1}{11}\begin{pmatrix} 4 & 3 \\ 1 & -2 \end{pmatrix}$.

In practice: `sl.solve(A, b)` solves the system (via an LU decomposition) *without* forming A^{-1}; prefer it to `sl.inv(A) @ b`. The two answers agree only to round-off.

5 The determinant

Full notes: *The determinant; Some more on the determinant; homework* → **used in L3, L4**

- Written $\det(A)$ or $|A|$; defined for square matrices. 2×2: $\begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc$. E.g. $\begin{vmatrix} 2 & 3 \\ 4 & 6 \end{vmatrix} = 0$ (singular).
- **Cofactor expansion** (along row 1; any row or column gives the same result): $|A| = \sum_j a_{1j}C_{1j}$, where the *minor* M_{ij} is the determinant of A with row i and column j removed, and the *cofactor* is $C_{ij} = (-1)^{i+j}M_{ij}$ (chessboard signs $+\ -\ +\cdots$). Applied recursively this defines $|A|$ for any n.
- **Properties:** $|AB| = |A|\,|B|$; $|A^T| = |A|$; $|\alpha A| = \alpha^n|A|$ for $n \times n$ A; a zero row or column gives $|A| = 0$; $|A| =$ product of the eigenvalues (L3); for a triangular matrix $|A| =$ product of the diagonal entries.
- **Geometry:** $|\det A|$ is the factor by which A scales areas (2D) or volumes; the sign records whether orientation is reversed. $\det A = 0$: space is squashed onto a lower dimension, so two different points map to the same point and A cannot be inverted.

Cramer's rule [**]

$$x_j = \frac{|B_j|}{|A|}, \qquad B_j = A \text{ with its } j\text{-th column replaced by } \boldsymbol{b}.$$

With recursive cofactor determinants this costs $\sim (n+1)!$, against $\mathcal{O}(n^3)$ for `sl.solve` (LU), so it is only usable for very small n. (Test system in the homework: $A = \left(\begin{smallmatrix} 2 & 3 & -4 \\ 6 & 8 & 2 \\ 4 & 8 & -6 \end{smallmatrix}\right)$, $\boldsymbol{b} = (5, 3, 19)^T$ gives $\boldsymbol{x} = (-6,\ 5,\ -\frac{1}{2})^T$.)

6 Solvability: independence, span, basis, rank and null space

Full notes: *Some linear algebra theory / solvability; Rank, range and null-space [*]* → **used in L3, L5, L6**

Linear independence	$\{\boldsymbol{v}_1, \dots, \boldsymbol{v}_k\}$ is linearly independent if $c_1\boldsymbol{v}_1 + \cdots + c_k\boldsymbol{v}_k = \boldsymbol{0}$ only for $c_1 = \cdots = c_k = 0$; otherwise one vector is a combination of the others (*dependent*).
Span	All linear combinations of the vectors (a line, a plane, …, through the origin).
Basis, dimension	A linearly independent set that spans the space. Every basis of $\mathbb{R}^n$ has n vectors (the *dimension*); the standard basis is $\boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k}, \dots$ (columns of I).
Range (column space)	$\text{range}(A) = \{A\boldsymbol{x}\}$, the span of the columns of A: the set of $\boldsymbol{b}$ for which $A\boldsymbol{x} = \boldsymbol{b}$ has a solution. The columns *span* it; they are a *basis* for it only if they are linearly independent.
Rank	$\text{rank}(A)$ = number of linearly independent columns = number of linearly independent rows $= \dim \text{range}(A)$. Square A: *full rank* if rank $= n$, else *rank deficient*.
Null space	$\text{null}(A) = \{\boldsymbol{x} : A\boldsymbol{x} = \boldsymbol{0}\}$. Always contains $\boldsymbol{0}$; closed under scaling.
Rank–nullity	For $A \in \mathbb{R}^{m\times n}$: $\text{rank}(A) + \dim \text{null}(A) = n$ (the number of columns).

Equivalent statements for a square $n \times n$ matrix A (invertible matrix theorem)

$\det A \neq 0 \iff$ the columns (equivalently the rows) are linearly independent $\iff$ the columns span $\mathbb{R}^n$, i.e. $\text{rank}(A) = n \iff \text{null}(A) = \{\boldsymbol{0}\} \iff A^{-1}$ exists $\iff A\boldsymbol{x} = \boldsymbol{b}$ has a unique solution for every $\boldsymbol{b}$.

- **If A is singular, $A\boldsymbol{x} = \boldsymbol{b}$ has no solution if $\boldsymbol{b} \notin \text{range}(A)$** (inconsistent equations), and **infinitely many** if $\boldsymbol{b} \in \text{range}(A)$ (consistent): any null-space vector can be added to a solution. Which case occurs depends only on $\boldsymbol{b}$. Example: $\left(\begin{smallmatrix}2&3\\4&6\end{smallmatrix}\right)\boldsymbol{x} = (4,8)^T$ has infinitely many solutions, e.g. $(2,0)^T$, $(-1,2)^T$; with $(4,7)^T$ it has none.
- **Computing rank and null space:** row operations do not change the rank, so reduce to (R)REF and count the non-zero rows; solve $A_{\text{RREF}}\boldsymbol{x} = \boldsymbol{0}$ for the null space. Example:

$$A = \begin{pmatrix} -2 & -4 & -20 \\ 2 & 6 & 24 \\ 2 & 10 & 32 \end{pmatrix} \xrightarrow{\text{row ops}} \begin{pmatrix} 1 & 0 & 6 \\ 0 & 1 & 2 \\ 0 & 0 & 0 \end{pmatrix} : \quad \text{rank}(A) = 2, \quad \text{null}(A) = \text{span}\{(6,2,-1)^T\} \text{ (dimension 1).}$$

NumPy/SciPy: `np.linalg.matrix_rank`, `sl.null_space`.

7 Linear transformations (geometry)

Full notes: *Geometric interpretation of linear transformations [*]* → **used in L3, L4**

- A map f is **linear** if $f(\boldsymbol{x}+\boldsymbol{y}) = f(\boldsymbol{x}) + f(\boldsymbol{y})$ and $f(\alpha\boldsymbol{x}) = \alpha f(\boldsymbol{x})$. In finite dimensions every linear map is multiplication by a matrix ($m \times n$ maps $\mathbb{R}^n \to \mathbb{R}^m$).
- $A\boldsymbol{p} = p_1(A\boldsymbol{i}) + p_2(A\boldsymbol{j})$: the basis vectors $\boldsymbol{i}, \boldsymbol{j}$ are mapped to the **columns of** A, and every point moves with them. Solving $A\boldsymbol{x} = \boldsymbol{b}$ asks how far to go along each column to reach $\boldsymbol{b}$; the columns need not be orthogonal, only independent (in 2D: neither is a multiple of the other).

Transformation	Matrix	Effect on the unit square
Shear	$\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$	Parallelogram of area 1 ($\det = 1$).
Rotation by θ (anticlockwise)	$\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$	Rotated square: $\det = 1$, columns orthonormal, lengths preserved.
General	$\begin{pmatrix} a & b \\ c & d \end{pmatrix}$	Parallelogram with sides $(a,c)^T$, $(b,d)^T$ and area $\lvert ad-bc\rvert$ (also the magnitude of their cross product).

8 Projection matrix (homework)

Full notes: *homework: projection matrix (cf. PCA and dimension reduction)* → **used in L3, L8**

For a **unit** vector $\boldsymbol{v}$ ($\boldsymbol{v}^T\boldsymbol{v} = 1$), the orthogonal projection of $\boldsymbol{u}$ onto the direction $\boldsymbol{v}$ is

$$\boldsymbol{u}_{\text{proj}} = (\boldsymbol{v}^T\boldsymbol{u})\,\boldsymbol{v} = (\boldsymbol{v}\boldsymbol{v}^T)\,\boldsymbol{u} = P\boldsymbol{u}, \qquad P = \boldsymbol{v}\boldsymbol{v}^T \ \text{(outer product, } n \times n),$$

and $\boldsymbol{d} = \boldsymbol{u} - P\boldsymbol{u}$ is orthogonal to $\boldsymbol{v}$. P is symmetric, has rank 1 and satisfies $P^2 = P$. For data stored one point per row ($N \times 2$ matrix X): $X_{\text{proj}} = (X\boldsymbol{v})\boldsymbol{v}^T$; one point per column ($2 \times N$): PX. (Note $\boldsymbol{v}\boldsymbol{v}^T$ is a matrix, not the scalar 1.)

9 Sensitive ("tricky") systems [*]

Full notes: *homework: tricky matrices* → **used in L4**

$$\begin{pmatrix} 6.073 & 2.693 \\ 4.772 & 2.116 \end{pmatrix} \boldsymbol{x} = \begin{pmatrix} 3.921 \\ 3.082 \end{pmatrix}: \qquad \det A \approx -5.3 \times 10^{-4}, \qquad \boldsymbol{x} \approx (5.66,\ -11.31)^T.$$

Changing a_{11} by -10^{-3} changes $\boldsymbol{x}$ by about 88% (relative, in norm); adding 10^{-3} to each entry of $\boldsymbol{b}$ changes it by about 21%; rounding the data to 2 decimal places changes it by about 97%. Such a nearly singular matrix makes the solution very sensitive to small changes in the data. A determinant is only "small" *relative* to something; the relative measure, the *condition number*, is introduced in L4.